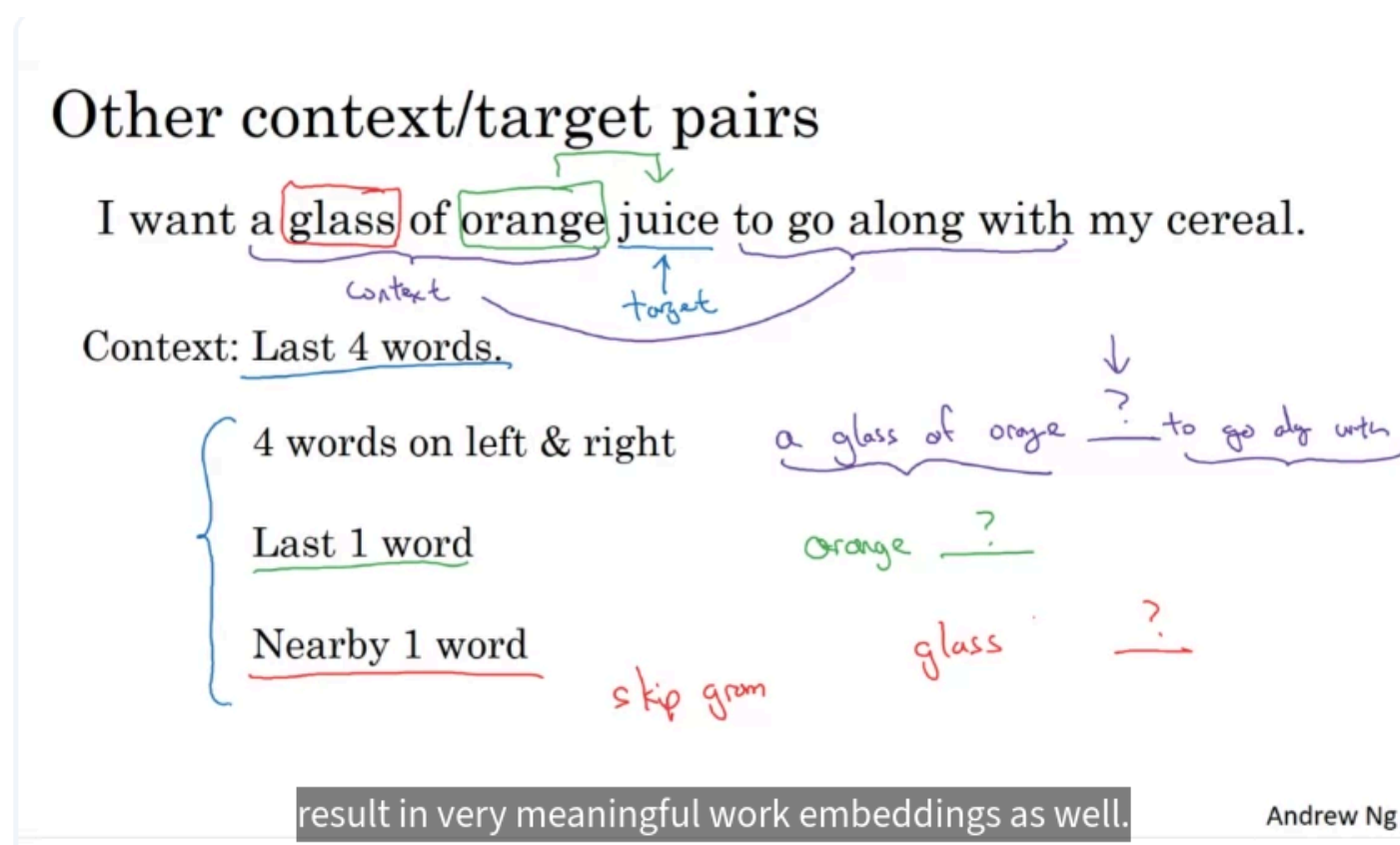
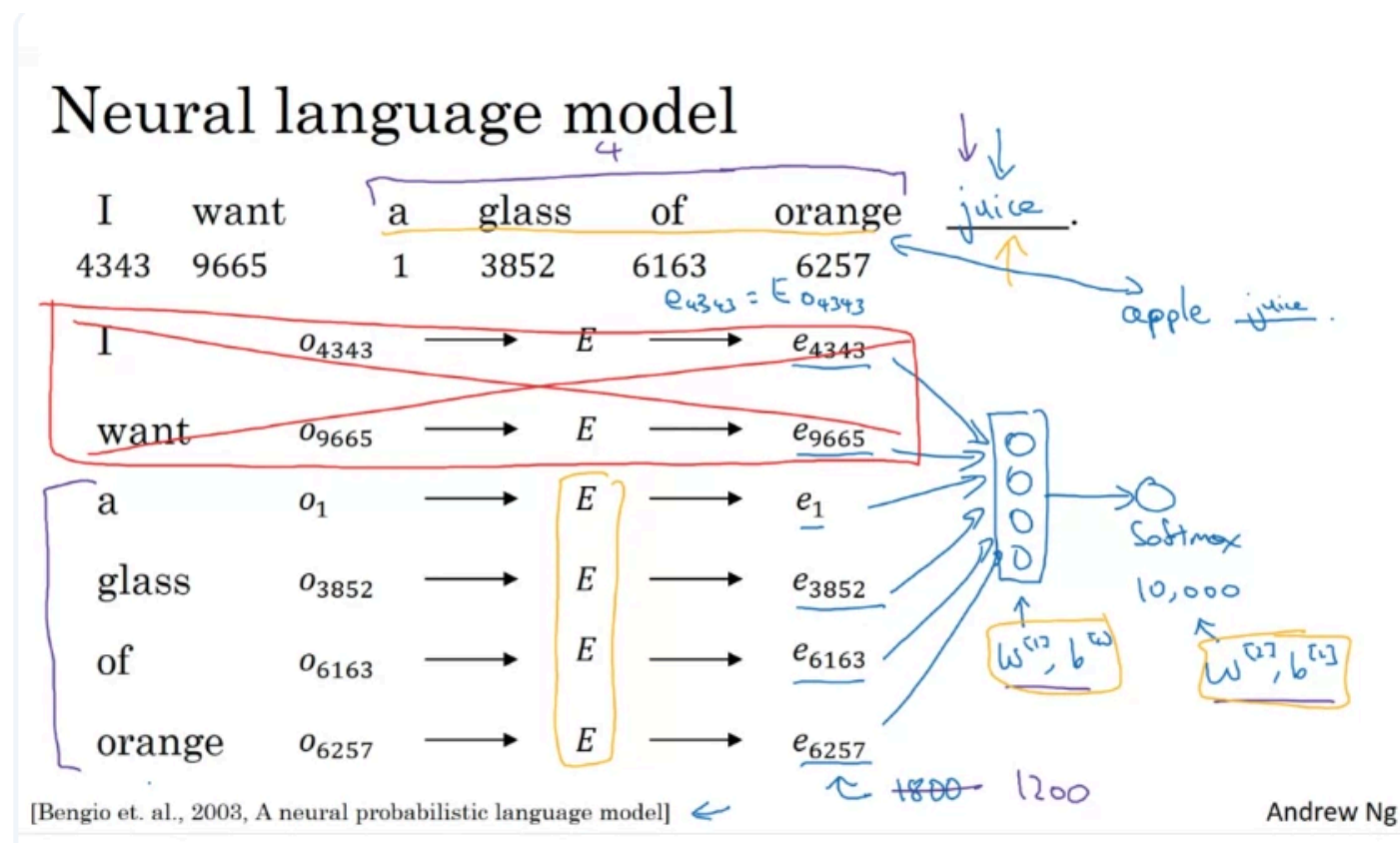


Learning Word Embeddings: Word2Vec & GloVe

Learning Word Embeddings

We have one hot vectors ($o_1, \dots$) then we multiply with Embedding layer, we got embedding vectors ($e_1, \dots$) and pass all of these vector to a neural language model $\rightarrow$ softmax $\rightarrow$ next word



Learning Word Embeddings with a Neural Language Model

- One of the earliest successful methods for learning word embeddings is to train a **neural language model**.
- Instead of learning embeddings directly, the model learns them **while predicting the next word** in a sentence.

Model Architecture

Input

- Each input word is first converted into a **one-hot vector**.
- The one-hot vector is multiplied by the **embedding matrix** (E) to retrieve its embedding.

Example: $e_i = E \cdot o_i$

where:

- (o_i) : one-hot vector
- (E) : embedding matrix
- (e_i) : word embedding

But this method getting vector has computational cost → just use index of the word to lookup

Neural Network

- The embeddings of the context words are:
 - Retrieved from the embedding matrix.
 - Concatenated into one long vector.
 - This vector is fed into:
 - A hidden layer
 - A softmax output layer
 - The softmax predicts the **target word** (next word in the sentence).
-

Fixed Context Window

Instead of using the entire sentence, the model typically uses a **fixed-size context window**.

Example:

Input:

| I want a glass

Predict:

| orange

or

Input:

| a glass of orange

Predict:

| juice

- Window size (e.g., 4 previous words) is a **hyperparameter**.
 - A fixed window keeps the input size constant regardless of sentence length.
-

Training the Model

The trainable parameters include:

- Embedding matrix **E**
- Hidden layer weights (**W₁**, **b₁**)
- Softmax weights (**W₂**, **b₂**)

Training is performed using **backpropagation and gradient descent** to maximize the probability of predicting the correct target word.

Why This Learns Good Embeddings

To predict words accurately, the model naturally learns similar embeddings for words appearing in similar contexts.

Example:

- orange juice
- apple juice
- grape juice
- pear juice

Since these fruits often appear before **juice**, their embeddings become similar.

Thus, semantic similarity emerges **without explicitly defining semantic features**.

Different Choices of Context

The language modeling objective can be modified depending on the goal.

Previous Words (Language Modeling)

Use the previous few words to predict the next word.

Example:

Context:

| a glass of orange

Target:

| juice

Surrounding Words

Use words on both the left and right to predict the middle word.

Example:

Context:

| a glass of orange ____ to go along with

Target:

| juice

Previous One Word

Use only the immediately preceding word.

Example:

Context:

| orange

Target:

| juice

Nearby One Word (Skip-Gram Idea)

Use one nearby word to predict another nearby word.

Example:

Context:

| glass

Predict a nearby word such as:

| orange

or

| juice

This simple idea becomes the foundation of the **Word2Vec Skip-Gram** model.

Language Modeling vs. Learning Embeddings

- If the goal is to build a **language model**, using several previous words as context is natural.
- If the goal is only to learn **high-quality word embeddings**, much simpler context definitions work surprisingly well.
- Researchers found that these simpler objectives can learn embeddings that are just as useful while being much more computationally efficient.

Key Takeaways

- Neural language models can learn word embeddings as a byproduct of predicting words.
- Every input word is mapped from a one-hot vector to a dense embedding through the embedding matrix.
- Words that occur in similar contexts naturally develop similar embeddings.
- The definition of "context" is flexible, and simpler context windows often produce equally effective embeddings.
- These observations led to simpler and more efficient methods such as **Word2Vec (Skip-Gram)**, introduced in the next section.

Word2Vec

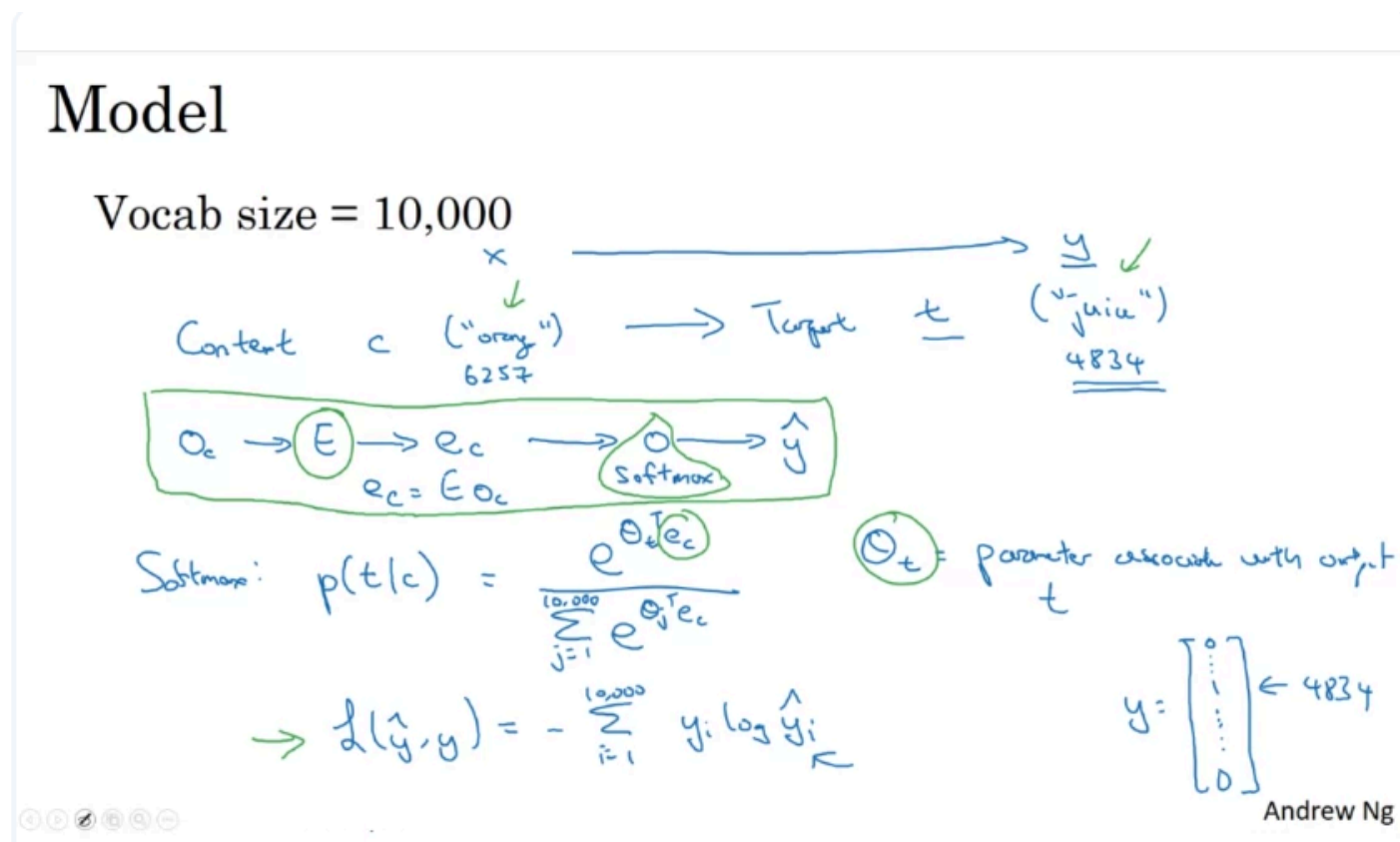
The video player displays a slide titled "Skip-grams". The slide content is as follows:

I want a glass of orange juice to go along with my cereal.

Handwritten annotations on the slide:

- Arrows point from the words "orange" and "juice" in the sentence to a handwritten "Context" label.
- Arrows point from the words "juice" and "glass" in the sentence to a handwritten "Target" label.
- Below the "Context" label, the word "orange" is listed three times, with an arrow pointing to the first instance.
- Below the "Target" label, the words "juice", "glass", and "my" are listed, with an arrow pointing to the first instance.

The video player interface at the bottom shows the title "[Mikolov et al., 2013: Efficient estimation of word representations in vector space.]", a progress bar at 2:07 / 12:33, a 1x speed setting, and the name "Andrew Ng".



Word2Vec: Skip-Gram Model

- **Word2Vec** is a simpler and more computationally efficient method for learning word embeddings.
- One important Word2Vec architecture is the **Skip-Gram model**.
- Its main goal is not necessarily to predict nearby words perfectly, but to use this prediction task to learn **high-quality word embeddings**.

Creating Training Pairs

Given a sentence, Skip-Gram creates **context–target pairs**.

Example sentence:

I want a glass of orange juice to go along with my cereal.

Suppose we randomly choose:

orange

as the **context word**.

Then we randomly select another word within a window around it, such as ± 5 or ± 10 words.

Possible training pairs:

Context	Target
orange	juice
orange	glass
orange	my

So the learning problem becomes:

Context word $\rightarrow$ Nearby target word

Why Random Nearby Words?

- A context word can have many possible nearby target words.
- Therefore, predicting the target is actually a difficult task.
- However, **accurate prediction itself is not the main objective**.
- The real purpose is to force the model to learn useful semantic representations.

Words that occur in similar environments will gradually receive similar embeddings.

Skip-Gram Model Architecture

Assume a vocabulary of:

$$V = 10,000$$

For a context word (c), such as **orange**:

Step 1: One-Hot Representation

Represent the context word using: o_c

For example, if **orange** is word #6257, then (o_c) is a 10,000-dimensional one-hot vector.

Step 2: Embedding Lookup

Use the embedding matrix (E): $e_c = E o_c$

where:

- (E) = embedding matrix
- (o_c) = one-hot context vector
- (e_c) = embedding of the context word

This operation simply retrieves the corresponding embedding vector.

Step 3: Softmax Prediction

The context embedding (e_c) is passed directly to a **softmax layer**.

The model predicts: $P(t|c)$

the probability that word (t) appears near context word (c).

For a target word (t):

$$P(t|c) = \frac{\exp(\theta_t^T e_c)}{\sum_{j=1}^V \exp(\theta_j^T e_c)}$$

where:

- (e_c): embedding of the context word
 - (θ_t): parameters associated with target word (t)
 - (V): vocabulary size
-

Training Target

The true target word is also represented using a **one-hot vector**.

For example, if:

| target = **juice**

and **juice** is word #4834: $y_{4834} = 1$

while all other elements are 0.

The softmax output: $\hat{y}$

contains predicted probabilities for every word in the vocabulary.

Loss Function

Skip-Gram uses the standard **softmax cross-entropy loss**: $L = - \sum_{i=1}^V y_i \log \hat{y}_i$

Since (y) is one-hot, this is equivalent to: $L = - \log P(t|c)$

So training tries to increase the probability of the actual nearby target word.

Parameters Learned

The model learns two main sets of parameters:

- **Embedding matrix (E)**
 - Contains the embeddings for context words.
- **Softmax parameters (θ)**
 - Used to predict target words.

By optimizing these parameters using gradient descent, the model learns useful word representations.

Why It Is Called Skip-Gram

The model takes one context word and tries to predict words that may occur **several positions away**.

For example:

| a glass of **orange** juice to go along with my

From **orange**, it might predict:

- glass
- juice
- my

Therefore, the model effectively **skips over words** when creating context–target pairs.

Key Takeaways

- **Skip-Gram** learns embeddings by predicting nearby words from a single context word.
- Training examples have the form:

Context → Nearby Target

- The basic architecture is very simple:

One-hot → Embedding lookup → Softmax → Target

- The prediction task is mainly a **proxy objective** for learning useful embeddings.
- Despite its simplicity, Skip-Gram can learn strong semantic representations and forms the foundation of **Word2Vec**.

Problems with softmax classification

$$p(t|c) = \frac{e^{\theta_t^T e_c}}{\sum_{j=1}^{10,000} e^{\theta_j^T e_c}}$$

Handwritten notes: *Hierarchical softmax.*, $\log |V|$, and a binary tree diagram showing the path from the root to the word "durian".

How to sample the context c ?

→ the, of, a, and, to, ...

→ orange, apple, durian

$P(c)$

t
 $c \rightarrow t$

Andrew Ng

Main Problem with Skip-Gram

- The main weakness of the basic Skip-Gram model is **computational cost**.
- The softmax probability is:

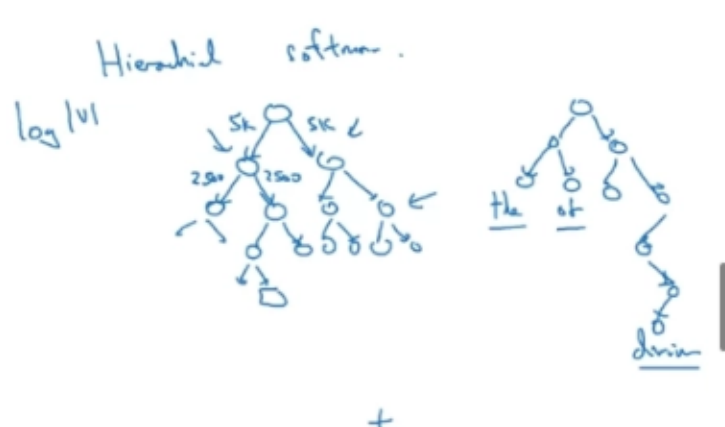
$$P(t|c) = \frac{\exp(\theta_t^T e_c)}{\sum_{j=1}^V \exp(\theta_j^T e_c)}$$

- The denominator requires summing over the **entire vocabulary**.
- This becomes expensive when the vocabulary contains:
 - 10,000 words
 - 100,000 words
 - 1,000,000+ words

So the computational cost grows roughly with: $O(V)$

where (V) is vocabulary size.

Hierarchical Softmax



One solution is **Hierarchical Softmax**.

Instead of directly choosing among all (V) words, words are organized in a **binary tree**.

Example for 10,000 words:

- Decide whether the target is in the first or second half.
- Decide which half of that subset it belongs to.

3. Continue until reaching the exact word.

Thus, prediction becomes a sequence of **binary classification decisions**.

Computational Advantage

With normal softmax: $O(V)$

With hierarchical softmax: $O(\log V)$

This makes training much faster for large vocabularies.

Tree Structure in Practice

- The tree does not necessarily need to be perfectly balanced.
- **Frequent words** can be placed closer to the root.
- **Rare words** can be placed deeper in the tree.

For example:

- **the**, **of**, **a** → shallow paths
- **durian** → deeper path

This is useful because frequent words are accessed much more often during training.

Sampling Context Words

Another important issue is how to choose the **context word (c)**.

A naive approach is to sample words directly according to their occurrence in the corpus.

However, extremely common words such as:

- the
- of
- a
- and
- to

would dominate the training data.

As a result, the model would spend too much time updating embeddings for common words and too little time learning embeddings for rarer but meaningful words.

Balancing Common and Rare Words

In practice, Word2Vec uses sampling heuristics to reduce the dominance of extremely frequent words.

The goal is to give sufficient training attention to less common words such as:

- orange
- apple
- durian

rather than having most training examples involve words such as **the** or **of**.

Two Main Word2Vec Architectures

The original Word2Vec work introduced two major architectures.

Skip-Gram

Use a center/context word to predict nearby words:

Center Word $\rightarrow$ Surrounding Words

Example:]

orange $\rightarrow$ juice

CBOW — Continuous Bag of Words

Use surrounding words to predict the middle word:

Surrounding Words $\rightarrow$ Center Word

Example:

| a glass of ? juice

Surrounding context:

| a, glass, of, juice

Target:

| orange

So Skip-Gram and CBOW essentially reverse the prediction direction.

Remaining Problem

Even with the simple Skip-Gram architecture, the standard softmax is still expensive because it requires evaluating all vocabulary words.

Therefore, an even more efficient training objective is needed.

One of the most important solutions is: Negative Sampling

which avoids computing the full softmax distribution.

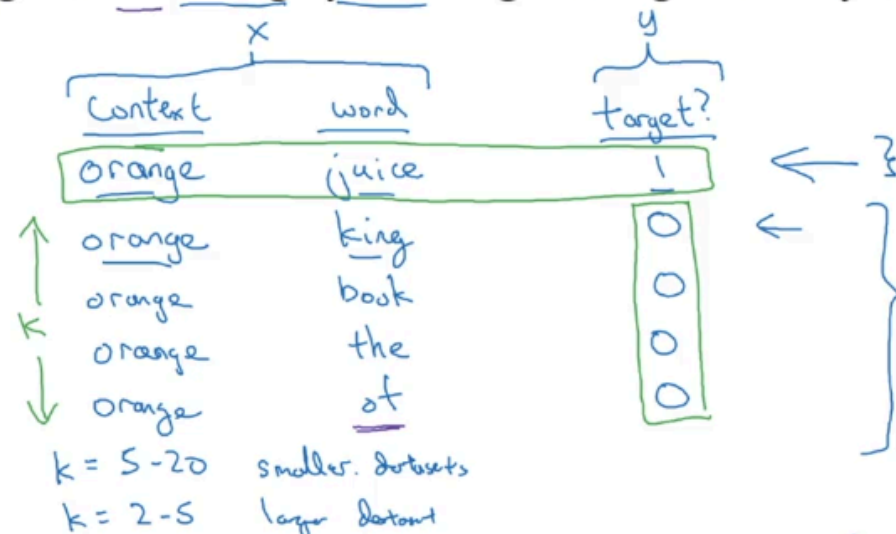
Key Takeaways

- Standard Skip-Gram with softmax is computationally expensive for large vocabularies.
- **Hierarchical Softmax** reduces complexity from approximately $O(V)$ to $O(\log V)$.
- Frequent words should not dominate the training process, so Word2Vec uses special sampling strategies.
- Word2Vec has two main architectures:
 - **Skip-Gram**: center $\rightarrow$ surrounding words
 - **CBOW**: surrounding words $\rightarrow$ center
- **Negative Sampling** provides another simpler and highly effective way to make Word2Vec training much more efficient.

Negative Sampling

Defining a new learning problem

I want a glass of orange juice to go along with my cereal.



[Mikolov et. al., 2013. Distributed representation of words and phrases and their compositionality]

Andrew Ng

Negative Sampling

- **Negative Sampling** is a more efficient alternative to the full softmax objective used in Skip-Gram.
- Instead of predicting a target word among the entire vocabulary, it converts the problem into a **binary classification task**.

New Supervised Learning Problem

Given a pair of words: (context, target)

the model predicts:

$$y = \begin{cases} 1 & \text{if the words form a real context-target pair} \\ 0 & \text{if the pair was generated randomly} \end{cases}$$

Example:

Context	Target	Label
orange	juice	1
orange	king	0
orange	book	0
orange	the	0
orange	of	0

Creating Positive Examples

Positive examples are generated in the same way as Skip-Gram.

1. Choose a **context word** from the corpus.
2. Choose a nearby word within a window such as ± 10 words.
3. Label the pair as: $y = 1$

Example: (orange, juice) $\rightarrow 1$

This means that **juice** was actually found near **orange** in the training corpus.

Creating Negative Examples

For each positive pair:

1. Keep the same context word.

2. Randomly sample (k) words from the vocabulary.
3. Pair each random word with the context word.
4. Label these pairs as: $y = 0$

Example:

(orange, king) $\rightarrow 0$

(orange, book) $\rightarrow 0$

These sampled words are called **negative samples**.

False Negative Samples Are Acceptable

- Sometimes a randomly sampled negative word may actually appear near the context word.
- For example: (orange, of) $\rightarrow 0$

even though **of** might occasionally appear near **orange**.

- This is acceptable because such cases are relatively rare and do not significantly harm training.
-

What the Model Learns

The model learns to distinguish between:

Real pairs

Words that actually occur near each other: $P(y = 1 \mid c, t)$

Example:

| orange — juice

Random pairs

Words generated independently:

$P(y = 0 \mid c, t)$

Example:

| orange — king

So the learning problem becomes:

Do these two words naturally occur together?

rather than:

Which of all V words is the target?

Number of Negative Samples (k)

For each positive example, generate (k) negative examples.

Typical recommendations:

- **Smaller datasets:** $k \approx 5 - 20$
- **Very large datasets:**

$k \approx 2 - 5$

Example with: k=4

gives:

- 1 positive pair

- 4 negative pairs

for each training example.

Why Negative Sampling Is Faster

With standard softmax, every training example requires computation over the entire vocabulary: $O(V)$

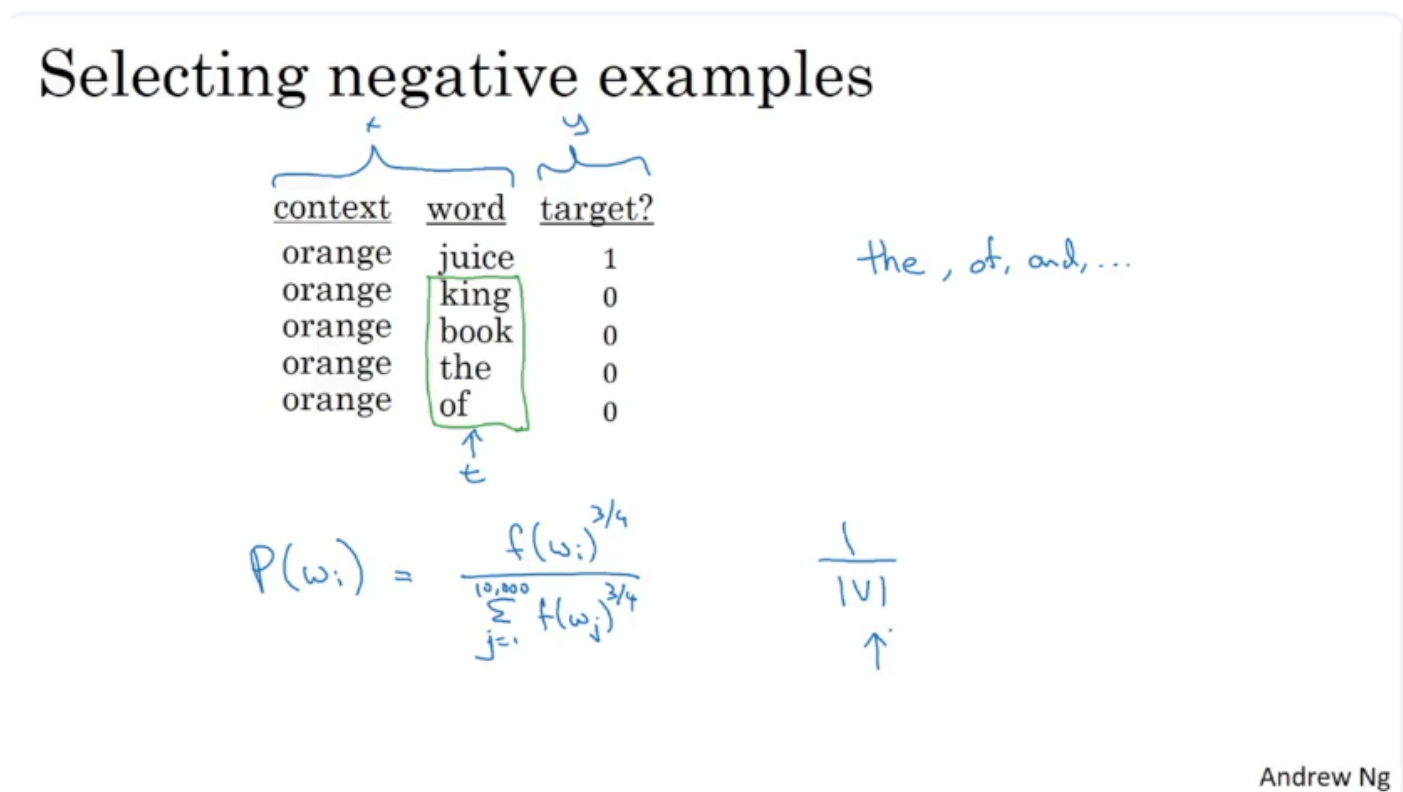
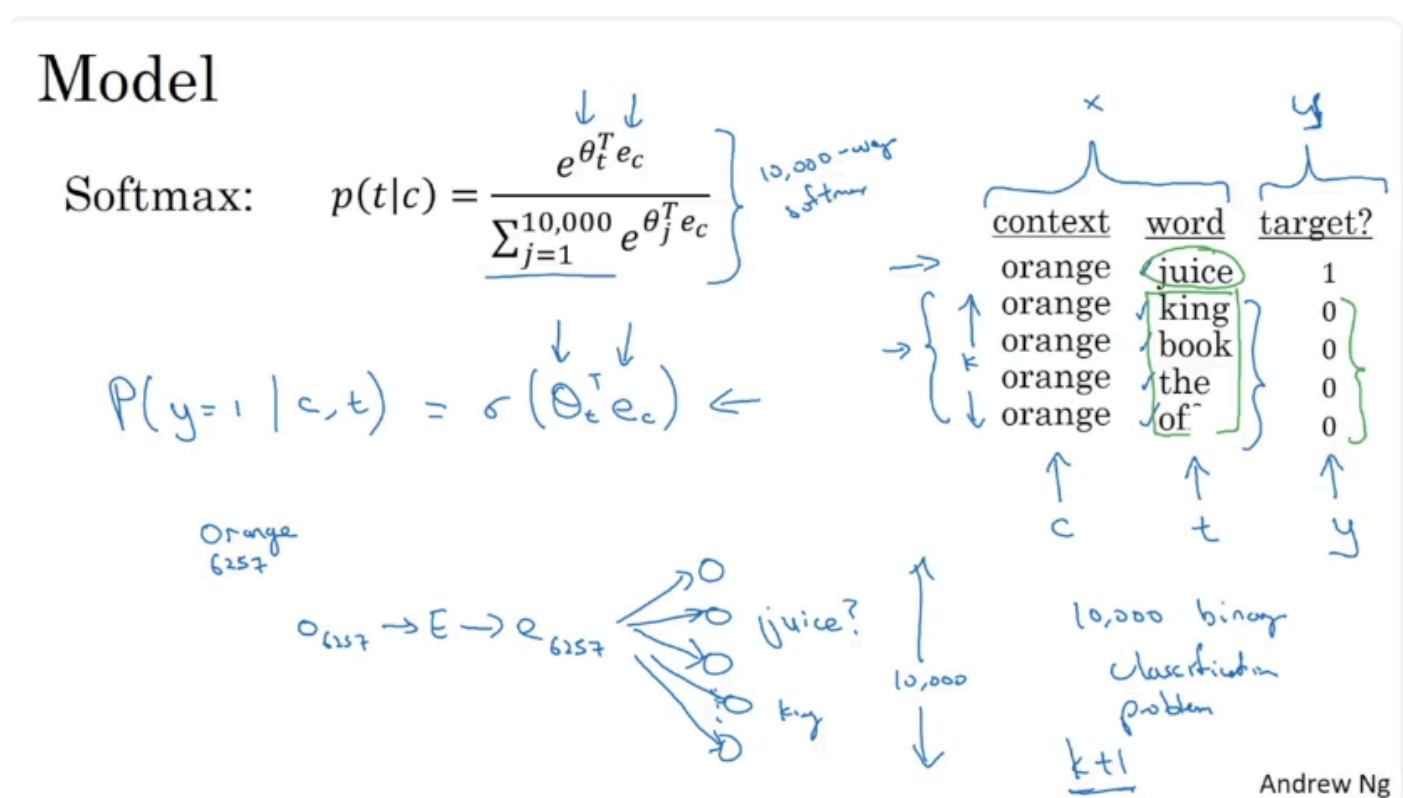
With Negative Sampling, the model only evaluates: $1 + k$

word pairs.

For example, with: $V = 1,000,000$, $k = 5$

instead of considering **1 million words**, the model only evaluates about **6 pairs** for that training example.

This makes Word2Vec dramatically more efficient on large corpora.



Negative Sampling as Binary Classification

- Negative Sampling replaces the expensive **multi-class softmax problem** with many small **binary classification problems**.
- For a context word (c) and candidate target word (t), the model predicts whether they form a real context-target pair:

$$P(y = 1 | c, t) = \sigma(\theta_t^T e_c)$$

where:

- (e_c) : embedding of the context word
- (θ_t) : parameter vector associated with the target word
- (σ) : sigmoid function
- $(y \in 0, 1)$

This directly models whether the pair is a genuine nearby pair or a randomly generated negative pair.

From One Large Softmax to Many Binary Classifiers

With a vocabulary of 10,000 words, we can think of the model as having:

$$10,000$$

possible binary logistic-regression classifiers.

For example, given the context word **orange**, there is conceptually a classifier for:

- Is the target **juice**?
- Is the target **king**?
- Is the target **book**?
- ...

However, the important point is that **we do not train all 10,000 classifiers on every step**.

If:

$$k = 4$$

then each training iteration updates only:

- 1 classifier for the positive target
- 4 classifiers for sampled negative targets

So only:

$$k + 1 = 5$$

binary classification problems are evaluated.

Why Negative Sampling Is Efficient

Standard softmax must calculate probabilities over the entire vocabulary.

Negative Sampling instead evaluates only:

$$1 + k$$

word pairs.

Therefore:

$$\text{Full Softmax: } O(V)$$

versus approximately:

$$\text{Negative Sampling: } O(k)$$

where typically:

$$k \ll V$$

This is the main computational advantage of Negative Sampling.

Training Example

Suppose the positive pair is:

$$(\text{orange}, \text{juice}), \quad y = 1$$

Then randomly sampled negative examples could be:

$$(\text{orange}, \text{king}), \quad y = 0$$

$$(\text{orange}, \text{book}), \quad y = 0$$

$$(\text{orange}, \text{the}), \quad y = 0$$

$$(\text{orange}, \text{of}), \quad y = 0$$

For (k=4), the model therefore trains on one positive pair and four negative pairs in that iteration.

How to Sample Negative Words

A key question is:

How should the negative target words be selected?

There are two obvious extremes.

Sampling by Corpus Frequency

We could sample according to how frequently words appear in the corpus:

$$P(w_i) \propto f(w_i)$$

Problem:

- Extremely frequent words such as:
 - the
 - of
 - and

would dominate the negative samples.

Uniform Sampling

Another option is:

$$P(w_i) = \frac{1}{V}$$

where every vocabulary word has the same probability.

Problem:

- This is too different from the actual distribution of natural language.
 - Very rare words would be sampled disproportionately often.
-

The (3/4) Sampling Heuristic

Word2Vec uses a compromise between these two extremes.

Negative words are sampled approximately according to:

$$P(w_i) = \frac{f(w_i)^{3/4}}{\sum_j f(w_j)^{3/4}}$$

where $f(w_i)$ is the observed frequency of word (w_i) .

So instead of: $f(w_i)$

we use: $f(w_i)^{3/4}$

This makes the distribution **flatter** than the original corpus distribution.

As a result:

- Very common words become less dominant.
 - Less frequent words receive relatively more probability.
 - But the sampling distribution still reflects real language frequencies.
-

Intuition for the (3/4) Power

Suppose:

$$f(\text{the}) = 10000$$

and:

$$f(\text{orange}) = 100$$

Using raw frequencies, the ratio is:

$$\frac{10000}{100} = 100$$

But after applying the (3/4) power:

$$\frac{10000^{3/4}}{100^{3/4}} 100^{3/4} \approx 31.6$$

So **the** is still sampled more frequently than **orange**, but **not 100 times more frequently**.

This balances frequent and less frequent words.

Parameters Learned

Negative Sampling still learns two sets of vectors:

- (e_c) : embeddings for context words
- (θ_t) : parameter vectors for target words

The model learns these parameters by determining whether: (c, t)

looks like a real word pair or a random pair.

After training, these learned vectors provide useful semantic word representations.

GloVe Word Vectors

GloVe (global vectors for word representation)

I want a glass of orange juice to go along with my cereal.

c, t

X_{ij} = # times i appears in context of j .

$X_{ij} = X_{ji}$ ←

Model

minimize $\sum_{i=1}^{10,000} \sum_{j=1}^{10,000} f(X_{ij}) (\theta_i^T e_j + b_i + b_j' - \log X_{ij})^2$

weighting term

$f(X_{ij}) = 0$ if $X_{ij} = 0$. "0 log 0" = 0

θ_i, e_j are symmetric

$e_w^{(final)} = \frac{e_w + \theta_w}{2}$

Example: $\theta_t^T e_c$ for "this, is, a, a, ..." and "durian"

We could read more here: [link](#)

GloVe: Global Vectors for Word Representation

- **GloVe** stands for **Global Vectors for Word Representation**.
- It was introduced by **Jeffrey Pennington, Richard Socher, and Christopher Manning**.
- Compared with Word2Vec, GloVe makes the idea of **word co-occurrence** more explicit.

Co-Occurrence Matrix

Define: X_{ij}

as the number of times word (i) appears in the context of word (j).

So:

X_{ij} = co-occurrence count of words i and j

- If context is defined symmetrically, such as words appearing within a ± 10 -word window:

$$X_{ij} = X_{ji}$$

- If context is directional, such as "the previous word", then in general:

$$X_{ij} \neq X_{ji}$$

Main Idea of GloVe

GloVe learns word vectors so that their dot product predicts how frequently two words occur together.

The simplified objective is:

$$\theta_i^T e_j \approx \log X_{ij}$$

Therefore, GloVe minimizes something like:

$$\sum_i \sum_j (\theta_i^T e_j + b_i + b'_j - \log X_{ij})^2$$

The intuition is:

- Words that frequently occur together should have embeddings whose **dot product reflects that relationship**.
 - Word relationships are therefore learned directly from **global co-occurrence statistics**.
-

Weighted Objective Function

The actual objective includes a weighting function:

$$J = \sum_i \sum_j f(X_{ij}) (\theta_i^T e_j + b_i + b'_j - \log X_{ij})^2$$

The weighting function ($f(X_{ij})$) controls how important each word pair is during training.

Handling Zero Co-Occurrence

If: $X_{ij} = 0$ then: $\log(0)$ is undefined.

Therefore, GloVe assigns zero weight to these pairs: $f(X_{ij}) = 0$

when: $X_{ij} = 0$

So word pairs that never co-occur are effectively ignored during optimization.

Why the Weighting Function Is Important

Word frequencies are highly unbalanced.

Very common words include:

- the
- is
- of
- a

Rare words might include:

- durian

Without weighting:

- Frequent words could dominate the objective.
- Rare but meaningful words might contribute too little.

The function ($f(X_{ij})$) tries to balance this:

- Frequent word pairs receive more importance, but **not excessively more**.
 - Less frequent pairs still receive meaningful training weight.
-

Symmetry Between (e) and (\theta)

An interesting property of GloVe is that: e_j and θ_i play essentially symmetric roles in the objective.

Unlike earlier Skip-Gram formulations, there is no strong conceptual distinction between "context embedding" and "target embedding".

Because of this symmetry, after training we can combine them.

For word (w):

$$e_w^{final} = \frac{e_w + \theta_w}{2}$$

This can be used as the final word embedding.

How GloVe Is Trained

1. Build a **word co-occurrence matrix** (X).
2. Initialize vectors (e) and (θ) randomly.
3. Optimize the weighted squared-error objective using gradient descent.
4. Optionally combine the two learned representations:

$$e_w^{final} = \frac{e_w + \theta_w}{2}$$

The resulting vectors become the word embeddings.

GloVe vs. Word2Vec

Word2Vec	GloVe
Learns from sampled context-target pairs	Learns from explicit co-occurrence counts
Prediction-based	Count/statistics-based
Skip-Gram predicts nearby words	GloVe models $(\log X_{ij})$
Uses objectives such as Negative Sampling	Uses weighted squared error

Despite the different objectives, both methods learn embeddings in which **semantically related words tend to have similar vectors**.

Key Takeaways

- GloVe explicitly uses **global word co-occurrence statistics**.
- Its central idea is:

$$\theta_i^T e_j \approx \log X_{ij}$$

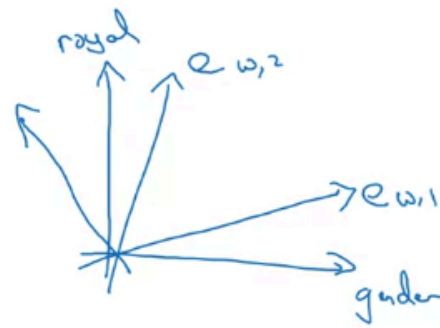
- It minimizes a weighted squared-error objective:

$$J = \sum_{i,j} f(X_{ij})(\theta_i^T e_j - \log X_{ij})^2$$

- The weighting function prevents extremely frequent words from dominating training.
- Context and target embeddings are symmetric and can be combined after training.
- Despite its simple objective, GloVe can learn meaningful semantic word representations.

A note on the featurization view of word embeddings

	Man (5391)	Woman (9853)	King (4914)	Queen (7157)	
Gender	-1	1	-0.95	0.97	←
Royal	0.01	0.02	0.93	0.95	←
Age	0.03	0.02	0.70	0.69	←
Food	0.09	0.01	0.02	0.01	←



$$\text{minimize } \sum_{i=1}^{10,000} \sum_{j=1}^{10,000} f(X_{ij}) (\theta_i^T e_j + b_i - b'_j - \log X_{ij})^2$$

$\underbrace{(\theta_i^T e_j)}_{(A\theta_i)^T (A^T e_j) = \theta_i^T A^T A^T e_j}$

Andrew Ng

Interpretability of Word Embedding Dimensions

- Earlier, we used an intuitive view where each embedding dimension represented a clear feature such as:
 - Gender
 - Royalty
 - Age
 - Food
- However, when embeddings are actually learned using algorithms such as **GloVe**, the individual dimensions usually **do not have clear human-interpretable meanings**.

Why Individual Dimensions Are Hard to Interpret

- The learning algorithm is free to choose any coordinate system in the embedding space.
- Therefore, the first dimension does not have to represent something specific like gender.
- Instead, one dimension may contain a mixture of many properties, such as:
 - Gender
 - Royalty
 - Age
 - Food
 - Cost
 - Size
 - Part of speech
 - Other semantic properties

So it is usually difficult to assign a clear meaning to a single embedding dimension.

Linear Transformation Intuition

Linear Transformation Intuition

Suppose an embedding model uses the interaction:

$$\theta_i^T e_j$$

If we apply an invertible transformation A , we can replace the vectors by transformed versions while preserving the same interaction:

$$\theta_i^T A^T A^{-T} e_j = \theta_i^T e_j$$

because:

$$A^T A^{-T} = I$$

This means many different coordinate systems can represent the same relationships between words.

Important Consequence

The embedding space may be:

- Rotated
- Stretched
- Transformed
- Or represented using non-intuitive axes

without changing the important relationships learned by the model.

Therefore:

individual dimensions are not necessarily meaningful

but:

relationships between vectors can still be meaningful

Analogy Relationships Still Work

Analogy Relationships Still Work

Even if individual dimensions are difficult to interpret, vector relationships such as:

$$e_{\text{woman}} - e_{\text{man}}$$

can still capture useful semantic directions.

Therefore analogy relationships such as:

$$e_{\text{woman}} - e_{\text{man}} \approx e_{\text{queen}} - e_{\text{king}}$$

can still hold.

This means the important information is often contained in the **geometry of the embedding space**, rather than in individual coordinates.

Key Takeaways

- Learned embedding dimensions are generally **not directly interpretable**.
- A single dimension may represent a mixture of many semantic properties.

- Different linear transformations can produce different coordinate systems while preserving the same underlying relationships.
- What matters most is the **relative geometry between vectors**.
- Semantic similarity and analogy relationships can still work even when individual embedding dimensions have no obvious meaning.